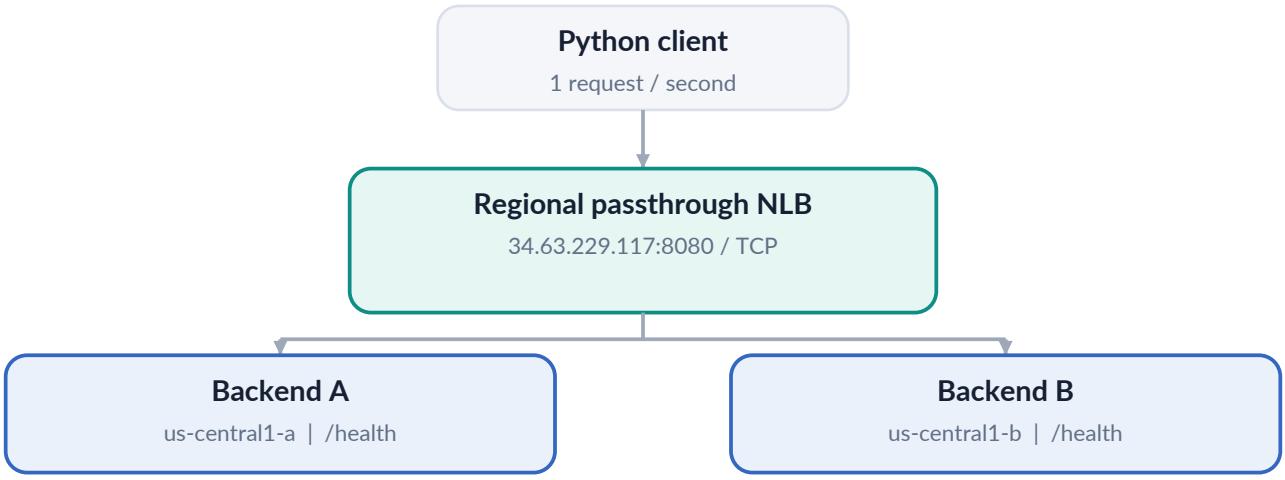
CS 528 / HOMEWORK 8

Zone-aware web serving with automatic failover

Two Google Compute Engine backends in different zones, fronted by a regional external passthrough Network Load Balancer.

30 / 30	50 : 50	19.6 s	24.7 s
Baseline OK	**Baseline ratio**	**Refusal window**	**Recovery**
0 request errors	15 A / 15 B	last refusal after stop	first A response

Architecture



Both server VMs run the same HW4-derived Flask/Gunicorn application. Every HTTP response adds the **X-Server-Zone** header, using the GCE metadata service. A dedicated **GET /health** endpoint returns HTTP 200 without fetching HTML from Cloud Storage.

Deployment configuration

Component	Actual configuration
Project / region	quick-formula-457319-q3 / us-central1
Backend A	hw8-server-a; us-central1-a; e2-micro; Debian 12; port 8080
Backend B	hw8-server-b; us-central1-b; e2-micro; Debian 12; port 8080
Backend groups	hw8-ig-a and hw8-ig-b; unmanaged, one VM each

01 / IMPLEMENTATION & REPRODUCTION

How the system is configured and run

Prerequisites. Authenticate the Google Cloud CLI to the target project. Reuse the existing storage bucket **gs://cloud_hw2_bucket/pages/** and the HW4 service account **hw4-web-sa** with permissions to read bucket objects; the inherited logging and Pub/Sub access preserves HW4 behavior. The setup script checks that the shared resources already exist.

1. Obtain the code and deploy. From the repository root, run:

```
git clone https://github.com/zukhriddingit/cs528-hw4-vm-services.git
cd cs528-hw4-vm-services/hw8
gcloud config set project quick-formula-457319-q3
bash scripts/setup.sh
```

2. What setup.sh provisions. It creates a temporary ingress rule allowing TCP:8080, two Debian 12 e2-micro VMs in separate us-central1 zones, zonal unmanaged instance groups, a regional HTTP health check, a regional TCP backend service, a reserved regional IP, and a TCP:8080 forwarding rule. Each VM receives the same Python source and requirements as startup metadata. The startup script installs dependencies, configures **hw8-server.service**, and launches Gunicorn.

3. Confirm both backends are healthy and test HTTP headers.

```
gcloud compute backend-services get-health hw8-backend --region=us-central1
LB_IP=$(gcloud compute addresses describe hw8-lb-ip \
  --region=us-central1 --format="value(address)")
curl -i "http://$LB_IP:8080/health"
curl -i "http://$LB_IP:8080/0.html"
```

The expected responses have status **200** and an **X-Server-Zone** header such as **us-central1-a** or **us-central1-b**. GET /0.html retrieves a 97,586-byte HTML file from Cloud Storage in the recorded test. HW3-style requests (GET, POST, unsupported methods, and forbidden-country simulation) remain available.

4. Run the zone-aware client. It starts one request per second, logs timestamp/zone/status/latency/errors to CSV, and prints the backend chosen by the load balancer. The commands below reproduce the measurement workloads:

```
python3 client.py --url "http://$LB_IP:8080/0.html" \
  --count 30 --out results/baseline.csv
python3 client.py --url "http://$LB_IP:8080/0.html" \
  --count 90 --out results/failover.csv
python3 client.py --url "http://$LB_IP:8080/health" \
  --count 75 --out results/recovery.csv
```

5. Failover / restoration commands (second terminal). While the corresponding client runs, stop the web service on A; later restart it. Use the VM clock in UTC as the change point.

```
gcloud compute ssh hw8-server-a --zone=us-central1-a \
  --command="sudo systemctl stop hw8-server; date -u +%FT%T.%3NZ"
gcloud compute ssh hw8-server-a --zone=us-central1-a \
  --command="sudo systemctl start hw8-server; date -u +%FT%T.%3NZ"
```

Stopping the **systemd service**, rather than killing a process, avoids automatic restart. The service remained stopped for the failover test.

02 / MEASUREMENTS

Traffic distribution and failover

All observations below were produced on **11 November 2026 UTC**. The client initiated requests approximately once per second. Its CSV timestamps represent **request start times**, not necessarily response completion or an internal load-balancer health-state transition. The initial baseline used **GET /0.html**.

Request counts by experiment



Run / endpoint	A complete	B complete	Errors / incomplete	A:B among complete
Baseline /0.html, 30	15	15	0	50.0% : 50.0%
Failover /0.html, 90	5	67	18	6.9% : 93.1%
Recovery /health, 75	15	60	0	20.0% : 80.0%

For the failover run, the original client printed **5 A / 69 B / 16 errors**. Inspection of the log shows that two B requests received HTTP 200 headers but timed out while reading the body. We classify those as **incomplete**, giving **5 A / 67 B / 18 failed or incomplete**.

Failover observation



Observed event	UTC timestamp	Relative to first stop
First stop of service A completed	02:11:20.938	0.000 s
First connection-refused attempt	02:11:21.553	+0.615 s
Last connection-refused attempt	02:11:40.552	+19.614 s
Next completed B response (request began)	02:11:41.551	+20.613 s
Later timeout still observed (request began)	02:12:28.772	+67.834 s

Interpretation. The last connection-refused error occurred **19.614 s** after the stop, indicating a roughly 20-second refusal window. Google Cloud subsequently showed A **UNHEALTHY** and B **HEALTHY**. Other request/read timeouts continued past 60 seconds, so the evidence does **not** show all errors ending within one minute. The cause of later timeouts was not isolated.

03 / RECOVERY & ANALYSIS

Restoration after the failure

After the failover run, the surviving server B answered both direct and load-balanced **/health** requests with HTTP 200. The recovery client then sent 75 GET /health requests while A was restarted.

02:20:43.901 UTC
Server A restarted

02:21:08.632 UTC
First A response

24.731 seconds observed

Recovery measurement	Observed value
Start command sent on VM A	02:20:43.853 UTC
systemctl start completed; service active	02:20:43.901 UTC
First subsequent A response (request start)	02:21:08.632 UTC
Observed return interval	24.731 s
Recovery request completion	75 / 75 successful, zero errors
Final backend health	A HEALTHY, B HEALTHY

From A's **first reappearance onward**, the final 28 requests were **15 to A** and **13 to B**, a **53.6% : 46.4%** split. This is the most representative recovery-period balance; the overall 15:60 split includes the time when only B was available.

Calculation and interpretation

The reported elapsed times use differences between UTC timestamps:

```
t_refusal_end = 02:11:40.552 - 02:11:20.938 = 19.614 s
t_first_A     = 02:21:08.632 - 02:20:43.901 = 24.731 s
ratio_A       = completed_from_A / completed_total
```

The load balancer distributes **TCP connections** using backend-selection hashing and connection tracking; it is not strict HTTP-request round robin. Ratios are observed samples and should not be interpreted as guaranteed 50/50 behavior. The first recovered-A request provides a client-observed reappearance time, not an exact internal health-check transition.

Reliability and limitations

- Two zones are an improvement over the single-VM HW4 design: a stopped web service on A eventually ceased receiving new connections, while B remained available.
- The failover workload fetched a 97,586-byte object through Cloud Storage. Timeouts could involve backend performance, network conditions, or client read deadlines; the experiment does not distinguish among them.
- For a more isolated health-check failover measurement, repeat failure testing against /health rather than /0.html and preserve a health-state polling timeline.
- The public TCP:8080 firewall rule (0.0.0.0/0) and ephemeral test infrastructure are suitable only for this controlled coursework experiment; clean up afterward.

04 / SOURCE, EVIDENCE & CLEANUP

Reproducibility and submission

The following GitHub links point directly to the Python code and scripts used to implement this solution:

HW8 code and documentation: Open on GitHub

Modified Python web server: Open on GitHub

Zone-aware Python client: Open on GitHub

VM and load balancer setup script: Open on GitHub

VM startup / systemd script: Open on GitHub

Infrastructure snapshot script: Open on GitHub

Safe infrastructure cleanup script: Open on GitHub

Recorded experiment summary: Open on GitHub

Evidence files

The raw measurement files are **baseline.csv** (30 GETs), **failover.csv** (90 GETs), and **recovery.csv** (75 GETs). They were retained locally under **hw8/results/** and are intended to accompany this report in the final GitHub submission so the summarized observations can be checked independently.

Terminal output also confirmed the state transitions: both backends were initially HEALTHY, then A UNHEALTHY / B HEALTHY, and finally both HEALTHY again. The first failover stop time 02:11:20.938 UTC was recorded from the initial stop command; a second redundant stop at 02:11:56.189 UTC did not define a new failure.

Save evidence, then remove billable resources

```
bash scripts/snapshot.sh
# Verify results/*.csv and the PDF report are saved before teardown.
bash scripts/cleanup.sh --yes
```

The cleanup script deletes only HW8-specific VMs/disks, instance groups, health check, backend service, forwarding rule, static IP, and firewall rule. It preserves the shared HW2 bucket, HW3 Pub/Sub resources, and HW4 service account. Historical usage may still appear on the GCP bill.

Outcome

The implementation met the core architectural goal: two serving VMs in different zones behind one regional load balancer, with a zone-identifying HTTP header and health-based rerouting. The client observed an approximately **20-second window ending the connection-refused failures** and a **24.731-second first-observed recovery**. The report discloses late failover timeouts instead of attributing them to health-check detection without evidence.

Technical references: Google Cloud setup guide; backend-service architecture; traffic distribution and connection tracking.